# Day 25: State Machines

**Platform:** nRF52840 | **RTOS:** Zephyr RTOS

---

## Learning Objectives

### enum class FSM

The simplest state machine uses `enum class` for states and a `switch` in a dispatch function:

```
enum class LedState : uint8_t { IDLE, BLINKING, ALARM };
enum class LedEvent : uint8_t { START, STOP, ALARM };

LedState current = LedState::IDLE;

void dispatch(LedEvent event) {
    switch (current) {
    case LedState::IDLE:
        if (event == LedEvent::START) {
            on_exit(current);
            current = LedState::BLINKING;
            on_entry(current);
        }
        break;
    // ...
    }
}
```

Entry/exit actions are called on every transition — keep them short.

### Template FSM Pattern

A template-based FSM moves the state transition table to compile-time data:

```
template <typename State, typename Event>
struct Transition {
```

```
    State   from;
    Event   trigger;
    State   to;
    void  (*action)(void);
};
```

The FSM iterates the transition table at runtime — but the table itself is in Flash ( `constexpr` ).

### Hierarchical State Machines

In a hierarchical FSM (HSM), states can have **parent states** that handle events not handled by the child. This reduces the number of transitions you have to define:

- State `FAST_BLINK` and `SLOW_BLINK` can both be children of `BLINKING`
- The `STOP` event is handled in `BLINKING` (the parent) — both children inherit it

### Zephyr SMF Framework

Zephyr includes a **State Machine Framework** ( `CONFIG_SMF=y` ) with built-in support for hierarchical states, entry/exit actions, and parent state event propagation:

```
static const struct smf_state my_states[] = {
    [IDLE]    = SMF_CREATE_STATE(idle_entry,   idle_run,   idle_exit, NULL, NULL),
    [ACTIVE]  = SMF_CREATE_STATE(active_entry, active_run, NULL, &my_states[IDLE], NULL),
};
smf_set_initial((struct smf_ctx *)&s, &my_states[IDLE]);
smf_run_state((struct smf_ctx *)&s);
```

### Event-Driven vs Polling FSM

- **Polling FSM:** `dispatch()` is called periodically from a timer — checks conditions and transitions. Simple but wastes CPU.
- **Event-driven FSM:** events arrive via Zephyr message queues, semaphores, or callbacks. The FSM thread blocks until an event arrives. More efficient.

### Guard Conditions and Actions

A **guard condition** is a boolean check that must be true for a transition to fire:

```
if (event == Event::BUTTON && battery_ok()) {  // Guard: battery_ok()
    transition_to(State::ACTIVE);
}
```

An **action** is code that executes during a transition (between `on_exit` and `on_entry`).

---

## Practice Tasks

1. Implement a 4-state traffic light FSM (RED → RED+YELLOW → GREEN → YELLOW → RED) using `enum class` and Zephyr timers
2. Convert your FSM to use Zephyr SMF — add an HSM with a parent state that handles the `RESET` event
3. Add guard conditions: the `FAST_BLINK` state only activates if battery voltage > 3.5V
4. Log every state transition with timestamp using `LOG_INF` — trace a sequence of events

---

## Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day25
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach   # RTT viewer (requires J-Link)
```

## Resources

- Zephyr Docs
- nRF52840 Product Specification

- Nordic DevZone
- Source: src/day25/main.cpp